

Software Requirement Specifications

[AidLedger]

Version: [1.8]

Project Code	F25-256
Supervisor	Dr. Shahbaz Siddiqui
Co Supervisor	Mr. Usama Antuley
Project Team	Muhammad Ali, Muhammad Usman, Abdul Moueed
Submission Date	Dec 4,2025

Document History

[Revision history will be maintained to keep a track of changes done by anyone in the document.]

Version	Name of Person	Date	Description of change
1.0	Abdul Moueed	11/13/25	Document Created
1.1	Abdul Moueed	11/19/25	Use case table for beneficiary Registration
1.2	Mohammad Ali	11/19/25	Use case table for Beneficiary Verification & Card Issuance
1.3	Mohammad Ali	11/23/25	Use case table for Beneficiary Verification & Card Issuance
1.4	Abdul Moueed	11/23/25	Use case table for Update Eligibility Policy
1.5	Muhammad Usman	11/25/25	Non-functional requirements
1.6	Muhammad Ali	11/27/25	External Interface Requirements
1.7	Abdul Moueed	11/29/25	Overall System Description
1.8	Muhammad Usman	12/03/25	Final review, formatting, and references added.

Distribution List

[Following table will contain list of people whom the document will be distributed after every sign-off]

Name	Role
Dr. Shahbaz Siddiqui	Supervisor
Mr. Usama Antuley	Co Supervisor

Document Sign-Off

[Following table will contain sign-off details of the document. Once the document is prepared and revised, this should be signed-off by the sign-off authority. Any subsequent changes in the document after the first sign-off should again get a formal sign-off by the authorities.]

Version	Sign-off Authority	Project Role	Signature	Sign-off Date
1.8	Supervisor	Supervisor	Dr. Shahbaz	04/12/2025

Table of Contents

1. Introduction.....	6
1.1 Purpose of Document.....	6
1.2 Intended Audience.....	6
1.3 Abbreviations.....	6
1.4 Document Convention.....	6
 2. Overall System Description.....	 7
2.1 Project Background.....	7
2.2 Project Scope.....	7
2.3 Not in Scope.....	8
2.4 Project Objectives.....	8
2.5 Stakeholders.....	8
2.6 Operating Environment.....	9
2.7 System Constraints.....	9
2.8 Assumptions & Dependencies.....	10
 3. External Interface Requirements.....	 11
3.1 Hardware Interfaces.....	11
3.2 Software Interfaces.....	11
3.3 Communications Interfaces.....	11
 4. Functional Requirements.....	 12
4.1 Functional Hierarchy.....	12
4.2 Use Cases.....	13
 5. Non-functional Requirements.....	 19
5.1 Performance Requirements.....	19
5.2 Safety Requirements.....	19
5.3 Security Requirements.....	19
5.4 User Documentation.....	19
 6. References.....	 20

1. INTRODUCTION

1.1. Purpose of Document

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed and structured outline of the functional and non-functional requirements for AidLedger, a decentralized Governance, Risk, and Compliance (GRC) framework. This document serves as a comprehensive guide for all stakeholders, including project supervisors, developers, and potential users, to ensure a shared understanding of the system's goals, functionalities, and constraints.

- To provide a clear communication framework among stakeholders.
- To act as a reference document for developers during the system's design, implementation, and testing phases.
- To ensure that the system's design and implementation align with the project objectives.
- To establish a foundation for requirement traceability throughout the FYP lifecycle.

1.2. Intended Audience

- Project Supervisors (Dr. Shahbaz Siddiqui, Mr. Usama Antuley)
- Project Jury Members
- Development Team (Muhammad Ali, Muhammad Usman, Abdul Moueed)
- Potential Users (Government Aid Agencies, Simulated Bank Staff)

1.3 Abbreviations

Abbreviation	Definition
GRC	Governance, Risk, and Compliance
SBP	State Bank of Pakistan
CNIC	Computerized National Identity Card
NADRA	National Database and Registration Authority
KYC	Know Your Customer
NFC	Near Field Communication
POS	Point of Sale
FYP	Final Year Project

1.4. Document Convention

- **Font Family:** Arial
- **Font Size:** 16 for headings, 12 for the rest of the content
- All requirements are listed in bullet points or numbered lists.

2. OVERALL SYSTEM DESCRIPTION

2.1. Project Background

Government aid distribution programs in Pakistan often face systemic issues such as misreporting, enrollment of ineligible beneficiaries, lack of real-time policy enforcement, and insufficient transparency.

AidLedger addresses these challenges by introducing a decentralized, blockchain-backed GRC framework. The system enforces eligibility rules through smart contracts, verifies beneficiary attributes using simulated NADRA data, and routes-controlled transactions through a dummy banking environment and prototype aid cards. The goal is to demonstrate a transparent, tamper-resistant model suitable for academic experimentation.

2.2. Project Scope

This system aims to demonstrate an end-to-end aid distribution workflow using decentralized technology, ensuring transparency, security, and policy-driven compliance.

Included Functionalities:

- **GRC Framework & Smart Contract Policies:** Development of a decentralized GRC structure aligned with simplified SBP regulatory principles, including rule versioning and policy activation workflows.
- **Beneficiary Registration & Verification:** Web-based portal for capturing demographic details, verification against simulated NADRA datasets, and automated eligibility evaluation via blockchain contract parameters.
- **Dummy Bank Operations:** Simulated bank interface for application review, KYC validation, approval management, and prototype AidCard generation (virtual/NFC).
- **Card & POS Transaction Demo:** POS interface for restricted purchases and controlled fund withdrawals, with real-time validation through blockchain contracts.
- **Urdu Notification System:** Automatic alerts to beneficiaries regarding approvals, re-evaluations, policy updates, and transaction outcomes.

2.3. Not In Scope

- Integration with real government infrastructures such as NADRA or SBP.
- Manufacture of EMV-compliant physical banking cards.
- Implementation of enterprise-grade fraud detection or biometric verification.
- Deployment to production-scale blockchain networks or multi-institution ecosystems.
- Full commercial UI/UX design beyond essential academic demonstration interfaces.

2.4. Project Objectives

The AidLedger system aims to achieve the following:

1. **Enhance Transparency & Accountability:** Provide immutable traceability of eligibility decisions and policy execution.
2. **Enforce Compliance via Smart Contracts:** Encode SBP-inspired rules as configurable on-chain parameters.
3. **Prototype End-to-End Aid Workflow:** Demonstrate registration → verification → approval → card issuance → controlled transactions.
4. **Support Dynamic Policy Updates:** Allow re-evaluation of beneficiaries whenever new policy versions are activated.
5. **Explore Feasible Prototype Card Technologies:** Implement virtual or NFC-based card mechanisms suitable for controlled demos.

2.5. Stakeholders

Stakeholder	Description
Beneficiaries	Individuals eligible to receive financial assistance under the simulated aid program.
Dummy Bank Staff	Simulated financial institution officers responsible for reviewing applications, issuing AidCards, and monitoring transactions.
System Administrators	Configure GRC policies, manage system data, and supervise beneficiary evaluations.
Developers (FYP Team)	Responsible for design, implementation, integration, and troubleshooting.
Supervisors & Jury	Academic evaluators reviewing technical compliance and prototype effectiveness.

2.6. Operating Environment

The system operates in a controlled academic/demo environment comprising:

- **Blockchain Layer:** Local Hyperledger-Besu private network connected with Hardhat for smart contract development and testing.
- **Backend Environment:** Server running Flask/Django or Node.js to expose REST APIs.
- **Frontend Portal:** Web-based interface accessible via standard browsers.
- **Card Hardware:** ACR122U NFC reader for NFC-based AidCard interactions (optional).
- **Client Devices:** Computer or mobile phones used by beneficiaries and bank staff.
- **Supporting Libraries:** web3, Solidity compiler, nfcpy/pyscard, and other required dependencies.

2.7. System Constraints

- **Blockchain Performance Constraints:** Smart contract execution and transaction confirmation depend on the underlying Ethereum node, introducing latency.
- **Prototype-Level Hardware Dependencies:** NFC reader functionality requires compatible drivers and specific OS configurations.
- **Limited Scalability:** The system is optimized for academic demonstration and not designed for high-traffic production loads.
- **Offline/Simulated Data Sources:** Real NADRA and SBP integrations are not available; validation relies on controlled datasets.
- **Prototype Security Constraints:** Only core security controls are implemented; advanced cyber-hardening is out of scope.

2.8. Assumptions & Dependencies

Assumptions

- Beneficiaries provide accurate and complete demographic information.
- Simulated NADRA datasets reasonably represent verification workflows.
- Users interacting with the portal possess basic digital literacy.
- Network and blockchain components will remain stable during demonstrations.

Dependencies

- **Blockchain Network:** Hyperledger-Besu for smart contract hosting and execution.
- **Backend Libraries:** web3.py, nfcpy, pycard, Flask/Django/Node.js.
- **Hardware:** NFC reader (ACR122U) for card demonstrations.
- **Database:** PostgreSQL for off-chain storage of profiles, policy versions, and transaction logs.

3. EXTERNAL INTERFACE REQUIREMENTS

3.1. Hardware Interfaces

- **NFC Reader (ACR122U):** Enables communication with prototype NFC-based AidCards, supporting read/write operations for card identity and encrypted metadata.
- **Server Hardware:** A standard workstation capable of running a blockchain node, backend APIs, and a local DBMS.
- **End-User Devices:** Desktop or mobile devices used by beneficiaries and dummy bank staff to access the portal.

3.2. Software Interfaces

- **Blockchain Node (Ethereum-Based):** Executes Solidity smart contracts, provides access to eligibility evaluation, and supports Web3 RPC for backend integration.
- **Backend Application Server:** Built using Flask/Django (Python) or Express (Node.js), exposing REST endpoints for all core functionalities.
- **Database System:** PostgreSQL with ACID-compliant transactional support for storing off-chain data.
- **Smart Contracts:** Deployed using Solidity to enforce policy logic for eligibility verification and transaction validation.

3.3. Communications Interfaces

- **Web3 Protocol:** Handles communication between backend server and Ethereum blockchain node using JSON-RPC calls.
- **HTTP/REST APIs:** Enable interaction between frontend portal, backend system, dummy bank interface, and POS module.
- **NFC Communication Protocols:** Used by the ACR122U reader to interact with NFC-based AidCards, supporting ISO14443 standard.

4. FUNCTIONAL REQUIREMENTS

4.1. Functional hierarchy

1. Beneficiary Management

- 1.1. Register Beneficiary (CNIC, Income, Household)
- 1.2. Verify Beneficiary via Simulated NADRA & KYC
- 1.3. View Beneficiary Eligibility Status

2. GRC & Policy Management

- 2.1. Encode SBP Eligibility Rules into Smart Contracts
- 2.2. Update Eligibility Policies (e.g., Income Threshold)
- 2.3. Automatically Re-evaluate Beneficiaries on Policy Update

3. Card Management

- 3.1. Issue Prototype Debit Card (Virtual/NFC)
- 3.2. Link Card to Blockchain-enforced Rules
- 3.3. Manage Card Status (Active/Inactive)

4. Transaction Management

- 4.1. Process Cash Withdrawal (within limits)
- 4.2. Process Grocery Purchase at Authorized Vendor
- 4.3. Validate Transaction against Blockchain Rules
- 4.4. Log All Transactions on Blockchain

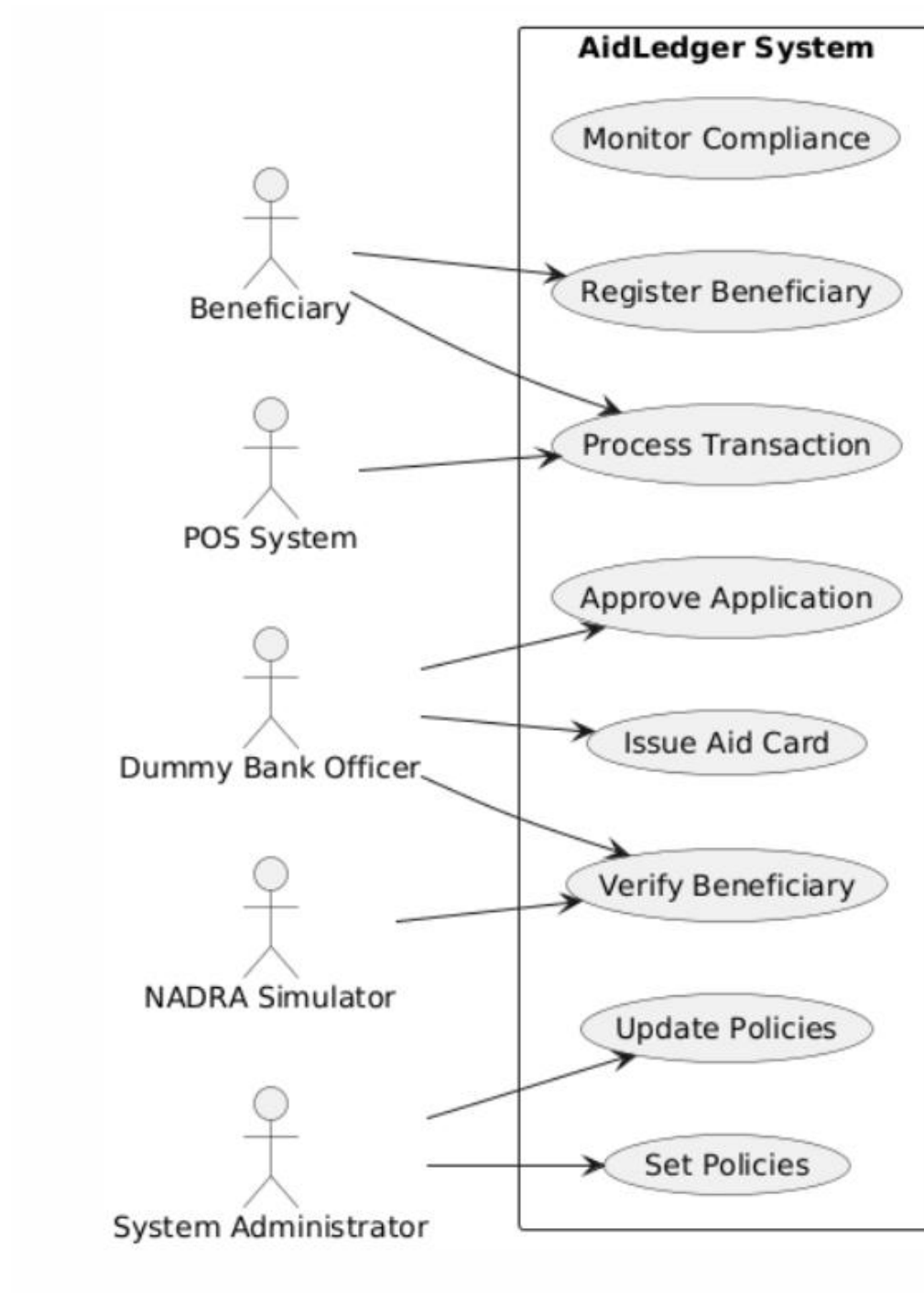
5. Notification Service

- 5.1. Generate Automated Notifications in Urdu
- 5.2. Notify on Eligibility Status Change
- 5.3. Notify on Policy Updates

6. Dummy Bank Operations

- 6.1. Simulate Bank KYC and Verification Process
- 6.2. Manage Card Issuance Workflow
- 6.3. View and Track Beneficiary Transactions

4.2. Use Cases



4.2.1. Beneficiary Registration (UC-BEN-01)

<UC-BEN-01>		
Use case Id:		<UC-BEN-01>
Actors:	Beneficiary: starts registration	
Feature:	Beneficiary Onboarding Module	
Pre-condition:		Beneficiary has valid CNIC and necessary information
Scenarios		
Step#	Action	Software Reaction
1.	Beneficiary accesses registration portal	Displays a registration portal
2.	Enters CNIC, income, household data	Software take input from beneficiaries
3.	Submits form for verification	Check if CNIC is valid then store it in database
Alternate Scenarios:		
1a: If beneficiary data is incomplete, the system prompts the Beneficiary to add missing information.		
2a: If invalid data is entered, the system displays an error message and prompts re-entry.		
3a: If CNIC already exists in the database, system displays User already registered message and redirects to Login.		
Post Conditions		
Step#	Description	
	Beneficiary is registered and pending verification	
Use Case Cross referenced		UC-BEN-01: Beneficiary registration

4.2.2. Beneficiary Verification & Card Issuance (UC-BANK-01)

<UC-BANK-01>		
Use case Id:		<UC-BANK-01>
Actors:	Dummy Bank Officer: starts verification	
Feature:	Identity Verification & Wallet Creation	
Pre-condition:		Beneficiary has completed registration
Scenarios		
Step#	Action	Software Reaction
1.	Bank officer reviews pending applications	System validates and processes request
2.	Triggers simulated NADRA/KYC check	System validates and processes request
3.	Approves eligible beneficiary and issues card	System records verification outcome
Alternate Scenarios:		
1a. Bank officer enters malformed CNIC number Displays error "Invalid CNIC format. Must be 13 digits."		
1b. System detects existing application for same CNIC Shows warning "Duplicate application found."		
Post Conditions		
Step#	Description	
	Verification recorded; balance updated	
Use Case Cross referenced		UC-BEN-01: Beneficiary registration UC-BANK-01: Beneficiary Verification & Card Issuance

4.2.3. Beneficiary groceries shopping (UC-TXN-01)

<UC-TXN-01>		
Use case Id:		<UC-TXN-01>
Actors:	Beneficiary, POS System: make and record transactions	
Feature:	Transaction Processing & Fund Transfer	
Pre-condition:		Beneficiary has active aid card with balance
Scenarios		
Step#	Action	Software Reaction
1.	Beneficiary presents card at authorized vendor	System processes payment authorization
2.	POS sends transaction to blockchain	System updates account balances
3.	Smart contract validates and approves transaction	System confirms transaction completion
Alternate Scenarios:		
2a.: Transaction amount exceeds available balance POS displays "Insufficient funds" Transaction rejected, no balance deduction 2b. Transaction would exceed daily spending limit Shows "Daily limit exceeded. Available: [amount]"		
Post Conditions		
Step#	Description	
	Transaction recorded on blockchain; balance updated	
Use Case Cross referenced		● UC-BEN-01: Beneficiary registration ● UC-BANK-01: Beneficiary Verification & Card Issuance ● UC-TXN-01: Beneficiary groceries shopping

4.2.4. Update Eligibility Policy (UC-ADMIN-01)

<UC-ADMIN-01>		
Use case Id:		<UC-ADMIN-01>
Actors:	Administrator: start updating policies	
Feature:	Policy Governance	
Pre-condition:		Admin is authenticated
Scenarios		
Step#	Action	Software Reaction
1.	Admin accesses policy management	System applies policy modifications
2.	Updates policy parameters	System applies policy modifications and triggers compliance recalculation
3.	System re-evaluates all beneficiaries	System updates beneficiary eligibility status
Alternate Scenarios:		
3a. Admin enters negative income threshold Displays "Invalid parameter: Income threshold must be positive."		
3b. User lacks policy modification permissions Shows "Access denied. Policy modification requires an admin role."		
Post Conditions		
Step#	Description	
	Policy updated; non-compliant beneficiaries flagged	
Use Case Cross referenced		● UC-BEN-01: Beneficiary registration ● UC-BANK-01: Beneficiary Verification & Card Issuance ● UC-TXN-01: Beneficiary groceries shopping ● UC-ADMIN-01: Update Eligibility Policy ●

4.2.5. Regulatory Audit (UC-BEN-01)

<UC-AUD-01>		
Use case Id:		<UC-AUD-01>
Actors:	Governing Body: Auditor	
Feature:	Governance & Oversight	
Pre-condition:		Network is synced; Transactions exist on the ledger.
Scenarios		
Step#	Action	Software Reaction
1.	Accesses the "Audit Dashboard" and sets filters (e.g., Transactions > 50,000 PKR).	System queries for matching transactions and displays the list.
2.	Selects a specific suspicious transaction ID.	System retrieves full transaction details from chain.
3.	Marks transaction as "Flagged for Investigation".	System records the "Flagged" status.
Alternate Scenarios:		
2a: Wallet Trace: System displays total inflow/outflow to detect money laundering patterns.		
Post Conditions		
Step#	Description	
	Suspicious activity report generated; Flagged status updated in off-chain database.	
Use Case Cross referenced		UC-TXN-01: Beneficiary groceries shopping

5. NON-FUNCTIONAL REQUIREMENTS

5.1. Performance Requirements

- **Response Time:** All critical transactions (e.g., POS validation, eligibility check) must complete within 5 seconds to ensure a smooth user experience, despite blockchain latency.
- **Throughput:** The system must be able to handle a minimum of 10 concurrent users for the academic demonstration environment.
- **Scalability:** The architecture must be designed to allow for future scaling of the backend and database layers.

5.2. Safety Requirements

- **Data Integrity:** All financial and eligibility data stored off-chain must be protected by ACID-compliant database transactions.
- **Blockchain Immutability:** The core GRC policies and transaction logs recorded on the blockchain must be immutable and tamper-proof.
- **Controlled Environment:** The system will operate in a controlled, simulated environment, mitigating risks associated with real-world financial systems.

5.3. Security Requirements

- **Authentication:** All users (Bank Staff, System Admins) must be authenticated via secure, password-protected login.
- **Authorization:** Role-based access control (RBAC) must be implemented to restrict access to sensitive functions (e.g., policy updates, card issuance) based on the user's role.
- **Data Confidentiality:** Sensitive beneficiary data (CNIC, income) must be encrypted both in transit (HTTPS/SSL) and at rest (database encryption).
- **Smart Contract Security:** Smart contracts must be audited for common vulnerabilities before deployment to the local network.

5.4. User Documentation

- The system must include comprehensive user manuals for the Dummy Bank Staff and System Administrators detailing the registration, verification, policy management, and transaction tracking processes.
- A brief guide for Beneficiaries on how to use the AidCard at a POS terminal must be provided.

6. REFERENCES

- **Hyperledger Besu Documentation** - Private Networks, IBFT 2.0 Consensus, and Permissioning (Official Documentation).
 - **ConsenSys Quorum Developer Guide** - Privacy Groups, Tessera Transaction Manager, and Zero-Knowledge Proof Integration.
 - **OpenZeppelin Contracts (v5.x)** - ERC-20 Token Standards, AccessControl, and Pausable Security Modules.
 - **Hardhat Development Environment** - Testing Frameworks, Gas Reporting, and Solidity Coverage Documentation.
 - **Benazir Income Support Programme (BISP) Annual Progress Report (2024-25)** - Digital Payment Mechanisms and Financial Inclusion Strategy.
 - **State Bank of Pakistan (SBP) Vision 2028** - Regulatory Framework for Digital Banks and Electronic Money Institutions (EMIs).
 - **NADRA e-Sahulat & Identity Stack** - API Documentation for Biometric Verification and CNIC Validation Services.
 - **BISP & World Food Programme (WFP)** - "Strengthening Social Protection for Nutrition in Pakistan" (2025 Report).
- Academic & Global Case Studies**
- **World Bank "FundsChain" Initiative** - Using Hyperledger Besu for Transparent Development Fund Tracking (Case Study).
 - **IEEE Xplore: Blockchain-Enabled Social Security Services Using Smart Contracts** - A framework for decentralized government aid distribution.
 - **UNICEF Innovation Fund** - Blockchain for Cash Transfer Programming in Humanitarian Aid (Technical Whitepaper).
 - **MDPI Applied Sciences: The Application of Blockchain in Social Media and Public Welfare Systems** (Systematic Review).

7. APPENDICES

- Not Applicable.